

and release cycle time and enable continuous delivery

- Organisations migrating from monolithic and legacy processes to microservices
- Scalability and flexibility of cloud services to handle more customers simultaneously without interrupting service
- Solutions that require the integration of business capability from disparate systems and programming models
- The need to align business and IT functionality
- Business and technology agility
- Better ROI
- The need for incremental development (Agile) and continuous deployment of business software

Benefits of microservices adoption

Microservices are built using lightweight frameworks and open source technologies. They offer a quick time to market and many other benefits too.

- Improve fault isolation; larger applications can remain largely unaffected by the failure of a single module.
- Provide business agility and support continuous delivery. New functionality can be added easily and quickly. Development teams work independently.
- Eliminate long-term commitment to a single technology stack; develop the code independently using whatever technology the developer feels is right for delivering business function. Different parts of systems can use different technology stacks if needed for better performance. Technology heterogeneity or polyglot programming is one essential benefit of microservices.
- Microservices make it easier for a new developer to understand the functionality of a service.
- Smaller codebases help developers focus and have a higher empathic relationship with the users of their

product, leading to better motivation and clarity in their work.

- Ease of deployment; one part of the system can move to another machine or to another cloud while the rest of the service can stay on-premise.

Best practices for the design and implementation of microservices

No legacy system can be transformed into microservices overnight. Organisations that want to introduce microservices must decide where and how these can be integrated into their existing applications. The first step is to define the microservices architecture. It has to be established how the services will interact with each other before a company attempts to optimise their implementation. Once microservices architecture gets going, we must be able to optimise the increase in speed.

It is better to start with a few coarse-grained but self-contained services. Fine graining can happen as the implementation matures over time. The developers, operations team, and testing fraternity may have extensive experience in monoliths, but a microservices-based system is a new reality; hence, they need time to cope with this new shift. Do not discard the monolithic application immediately. Instead, have it co-exist with the new microservices, and iteratively deprecate similar functionalities in the monolithic application.

This is not easy and requires a significant investment in people and processes to get started. As with any technology, it is always better to avoid the big bang approach, and identify ways to get the toes wet before diving in head first.

Always adopt automated toolsets that need to be added to the stack. Granular monitoring, log aggregation and dashboard, application metrics, automated deployment and system dashboard need to be addressed as part of the automated toolset.

The best practices for designing and implementing microservices are briefly listed below.

Business capability based design:

Microservices architecture typically needs teams that specialise in business capabilities with different requirements, such as order approval, shipping, delivery and returns management, which are all part of order fulfilment. Companies should aim to focus not on more projects, but on products. Services should be developed as independent products that are well documented and each is only responsible for a single business capability.

New features: Identify new features that must be added to the monolithic application and develop them instead as microservices. Alternatively, if new features cannot be identified, it is desirable to replace features in the monolithic application that are 'less critical' and more 'self-contained'.

New microservice: Agree on a standard interface for communication between the monolithic application and the new microservice. The new microservice needs to be self-contained, and hence, has to have its own runtime environment. The service also needs to be deployed on a separate infrastructure that is isolated, either physically or logically, from the monolithic application.

Granularity: Agree on building coarse-grained services that are more actively or frequently used, rather than a bunch of fine-grained services. The coarse-grained services will lead to fewer network calls being made between the monolithic application and the microservices.

Separate data store for each microservice: Do not use the same data store for all microservices. Each microservices team should select the database that is best suited to its service. A single data store also holds a significant risk that microservices from different teams share the same database structures, perhaps to avoid duplicating